

DEPARTMENT OF CSE (CYBERSECURITY)

U23CC583- DIGITAL AND MOBILE FORENSICS RECORD

SRI ESHWAR COLLEGE OF ENGINEERING

KINATHUKADAVU, COIMBATORE – 641202

SRI ESHWAR COLLEGE OF ENGINEERING

KINATHUKADAVU, COIMBATORE-641202

(An Autonomous Institution Affiliated to Anna University, Chennai)

DEPARTMENT OF CSE (CYBERSECURITY)

BONAFIDE CERTIFICATE

Certified that this is the bonafide record of work done by Mr./ Ms ……..……

…… Register No : ……..... of III Year **B.E.**

……..……in

the **U23CC583- Digital and Mobile Forensics** during the **5th Semester** of the academic year **2026- 27 (Odd Semester)**

Signature of Staff In-charge **Head of the Department**

Submitted for the End Semester Practical Examinations, held on…….

Internal Examiner **External Examiner**

LIST OF EXPERIMENTS

EX. NO.	DATE	NAME OF THE EXPERIMENT	PAGE NO.	MARKS (75)	SIGN
1	25/06/2026	Implementation of Metadata Extraction from a Digital Image			
2	16/07/2026	Recovery and Analysis of Deleted Files from a USB Drive			
3	13/08/2026	Implementation of Log Monitoring for Forensic Readiness			
4	03/09/2026	Analysis of Data Extraction from an iOS Backup			
5	01/10/2026	Implementation of Data Extraction from an Android Device using ADB			

Signature: **Average:**

Ex. No: 1 Date: 25/06/2026	Implementation of Metadata Extraction from a Digital Image

Aim:

To extract and analyze metadata (EXIF data) from a digital image using the ExifTool command-line utility, while following the standard Digital Forensics Investigation Process to preserve the integrity of the digital evidence.

Requirements:

Hardware

- Computer/Laptop
- Digital image file (JPG/JPEG captured using a smartphone or digital camera)

Software

- Windows Operating System
- ExifTool (command-line utility, downloaded from exiftool.org)
- Command Prompt

Theory:

Metadata is "data about data". Every digital image may carry hidden information embedded inside the file in addition to the visible picture. The most common standard is **EXIF (Exchangeable Image File Format)**, which is written automatically by cameras and smartphones. EXIF metadata can include the camera make and model, date and time of capture, exposure settings, software used, image resolution, and GPS coordinates of the place where the photo was taken.

In digital forensics, image metadata helps an investigator to:

- Identify the device used to capture the image.
- Establish a timeline (when the image was created, modified or edited).
- Determine the location of capture using GPS tags.
- Detect tampering, since editing software often leaves its name or changed timestamps in the metadata.

To preserve evidence integrity, the investigator should always work on a **copy** of the original image and record its hash value (MD5/SHA-256) before and after analysis.

Procedure:

Step 1: Install ExifTool

- Download the Windows executable ZIP of ExifTool from exiftool.org.
- Extract the ZIP file and rename exiftool(-k).exe to exiftool.exe.
- Copy exiftool.exe (along with the exiftool_files folder) into the working folder, for example C:\MetadataLab.

Verify the installation by opening Command Prompt in that folder and running:
exiftool -ver

```
C:\Users\raghu\Downloads\exiftool-13.59_64\exiftool-13.59_64>exiftool -ver
13.59
```

Step 2: Prepare the Image

- Take a photograph using a smartphone/camera (keep location enabled to obtain GPS data) and copy it to the computer.
- Save a copy of the image as sample.jpg in C:\MetadataLab.

- Do **not** edit or re-save the original image.

Step 3: Calculate the Hash Value (Integrity Check)

- Open Command Prompt and run:
- Extracts all available metadata from the image.
- exiftool sample.jpg

Step 4: Extract All Metadata

- exiftool sample.jpg

```
C:\Users\raghu\Downloads\exiftool-13.59_64\exiftool-13.59_64>exiftool sample.jpg
ExifTool Version Number         : 13.59
File Name                       : sample.jpg
Directory                       : .
File Size                       : 20 kB
Zone Identifier                 : Exists
File Modification Date/Time     : 2026:08:13 11:50:03+05:30
File Access Date/Time           : 2026:10:09 15:36:35+05:30
File Creation Date/Time         : 2026:08:13 11:50:06+05:30
File Permissions                : -rw-rw-rw-
File Type                       : JPEG
File Type Extension             : jpg
MIME Type                       : image/jpeg
JFIF Version                    : 1.01
```

```
Resolution Unit                 : inches
X Resolution                    : 300
Y Resolution                    : 300
Image Width                     : 525
Image Height                    : 350
Encoding Process                : Progressive DCT, Huffman coding
Bits Per Sample                 : 8
Color Components                : 3
Y Cb Cr Sub Sampling            : YCbCr4:2:0 (2 2)
Image Size                      : 525x350
Megapixels                      : 0.184
```

This displays every metadata tag found in the image.

Step 5: Extract Metadata Grouped by Category

- exiftool -a -u -g1 sample.jpg

```
C:\Users\raghu\Downloads\exiftool-13.59_64\exiftool-13.59_64>exiftool -a -u -g1 sample.jpg
---- ExifTool ----
ExifTool Version Number         : 13.59
Warning                         : [minor] Unknown APP1 segment
---- System ----
File Name                       : sample.jpg
Directory                       : .
File Size                       : 20 kB
Zone Identifier                 : Exists
File Modification Date/Time     : 2026:08:13 11:50:03+05:30
File Access Date/Time           : 2026:10:09 15:36:47+05:30
File Creation Date/Time         : 2026:08:13 11:50:06+05:30
File Permissions                : -rw-rw-rw-
---- File ----
File Type                       : JPEG
File Type Extension             : jpg
MIME Type                       : image/jpeg
Image Width                     : 525
Image Height                    : 350
Encoding Process                : Progressive DCT, Huffman coding
Bits Per Sample                 : 8
Color Components                : 3
Y Cb Cr Sub Sampling            : YCbCr4:2:0 (2 2)
---- JFIF ----
JFIF Version                    : 1.01
Resolution Unit                 : inches
X Resolution                    : 300
Y Resolution                    : 300
---- Composite ----
Image Size                      : 525x350
Megapixels                      : 0.184
```

The -g1 option groups the tags by category (File, EXIF, GPS, XMP, etc.), -a shows duplicate tags and -u shows unknown tags.

Step 6: Extract File Information

- exiftool -FileName -FileSize -FileType -MIMEType -ImageSize -FileCreateDate -FileModifyDate -FileAccessDate sample.jpg

```
C:\Users\raghu\Downloads\exiftool-13.59_64\exiftool-13.59_64>exiftool -FileName -FileSize -FileType -MIMEType -ImageSize -FileCreateDate -FileModifyDate -F
leAccessDate sample.jpg
File Name                       : sample.jpg
File Size                       : 20 kB
File Type                       : JPEG
MIME Type                       : image/jpeg
Image Size                      : 525x350
File Creation Date/Time         : 2026:08:13 11:50:06+05:30
File Modification Date/Time     : 2026:08:13 11:50:03+05:30
File Access Date/Time           : 2026:10:09 15:38:18+05:30
```

Step 7: Save the Metadata to a Report File

- exiftool sample.jpg > metadata_report.txt
- exiftool -csv sample.jpg > metadata_report.csv

Step 8: Analyze the Metadata

Inspect and interpret the following:

- Make and Model: identifies the capturing device.
- DateTimeOriginal vs. FileModifyDate: a mismatch may indicate that the file was edited or copied.
- Software: shows whether an editing tool (e.g., Photoshop) was used.
- GPS coordinates: the location where the image was taken.

Step 9: Re-check Integrity

- certutil -hashfile sample.jpg SHA256

```
C:\Users\raghu\Downloads\exiftool-13.59_64\exiftool-13.59_64>certutil -hashfile sample.jpg SHA256
SHA256 hash of sample.jpg:
2639025776a505aa4d43d80998a35f91cddbba591d5877bd7ea3b0685f7e6a94
CertUtil: -hashfile command completed successfully.
```

Confirm that the hash value matches the one noted in Step 3, which proves that the evidence was not altered during analysis.

Step 10: Document the Findings

Record the observations in the table below.

Parameter	Value Observed
File name	Sample.jpg
File size	20 KB
File type / MIME type	JPEG
Image size	525 x 350
Camera make & model	Samsung S25 Ultra
Date & time of capture	2026:10:09 15:36:35+05:30
Software used	EXIFTOOL
GPS latitude & longitude	-
SHA-256 (before / after)	-

Result:

The metadata of the digital image, including file attributes, EXIF details and GPS information, was successfully extracted and analyzed using ExifTool commands, and the integrity of the evidence was maintained through hash verification.

SRI ESHWAR COLLEGE OF ENGINEERING		
PARAMETERS	**MAX MARKS**	**MARKS OBTAINED**
Pre Lab	20	
Program	25	
Output	20	
Viva	10	
Total	75	
SIGNATURE OF THE FACULTY		

Ex. No: 2 Date: 16/07/2026	Recovery and Analysis of Deleted Files from a USB Drive

Aim:

To recover and analyze deleted files from a USB flash drive using the Autopsy Digital Forensics Platform while following the standard Digital Forensics Investigation Process to preserve the integrity of digital evidence.

Requirements:

Hardware

- Computer/Laptop
- USB Flash Drive (8 GB or above)

Software

- Autopsy Digital Forensics Platform [Java (if required by your Autopsy version)]
- Windows Operating System

Theory:

When a file is deleted from a USB drive, the operating system generally removes only the file system reference, while the actual data remains stored on the device until it is overwritten by new data. Digital forensic tools such as Autopsy can identify these deleted file entries, recover recoverable files, analyze their metadata, and generate forensic reports.

Procedure:

Step 1: Download Autopsy Step

Step 2: Prepare the USB Drive

Before starting:

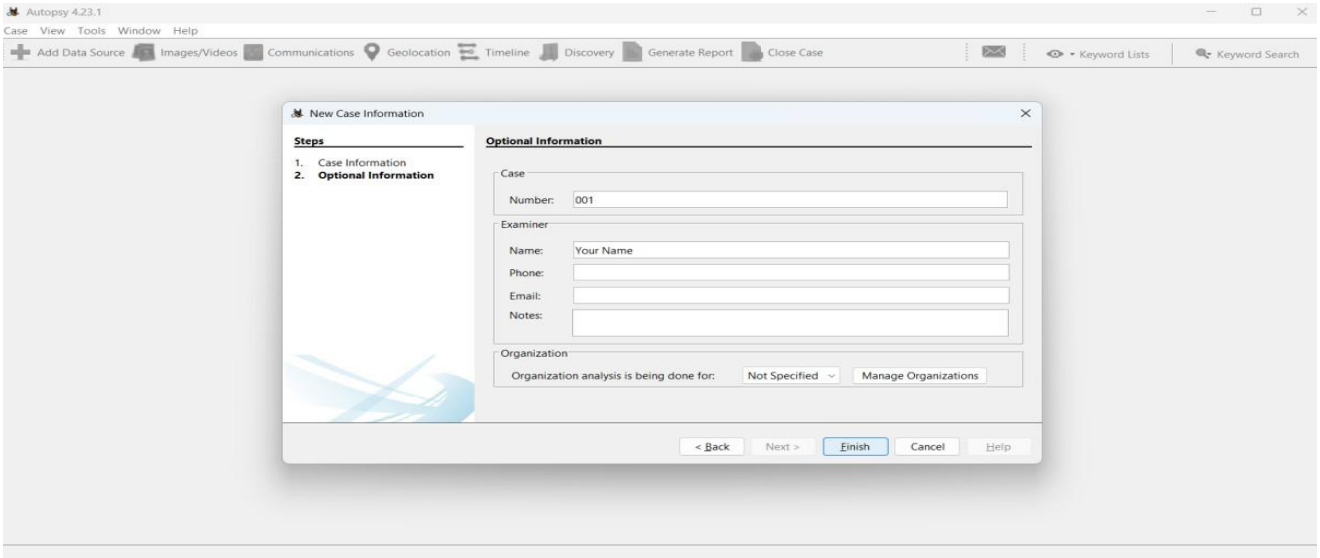
- Copy a few files (PDF, JPG, DOCX, TXT) onto a USB drive.
- Delete those files.
- Do **not** copy any new files to the USB after deletion. This ensures that deleted data has not been overwritten.

Step 3: Connect the USB. Insert the USB drive into your computer

- Do not format it.
- Do not save anything new to it.

Step 4: Open Autopsy

- Click: Create New Case

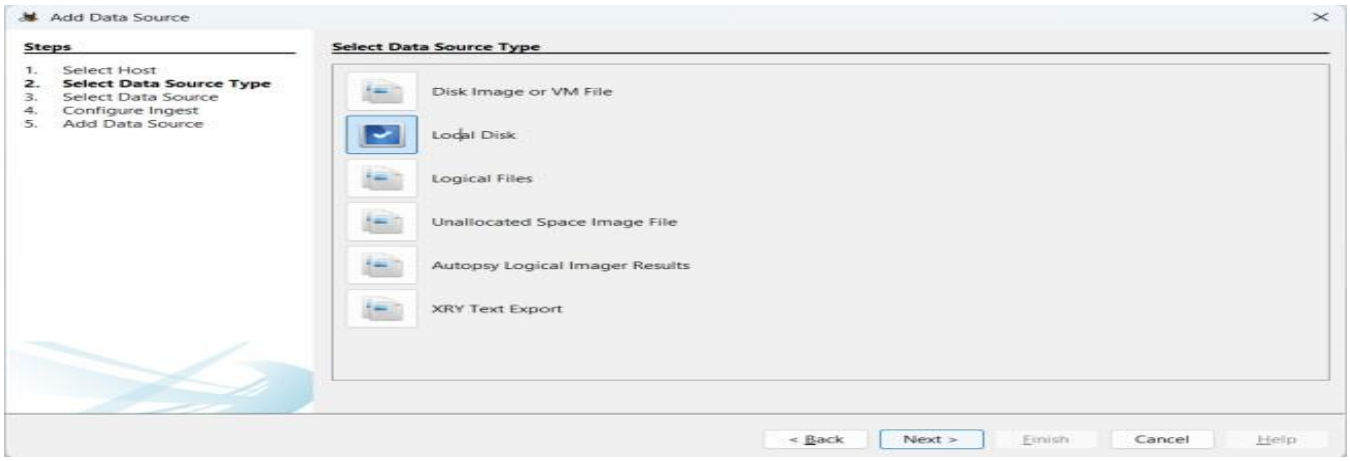


- Enter Case Name: USB Deleted File Recovery

- Base Directory: Choose a folder where Autopsy will save the case.
- Examiner Name: Your Name
- Case Number: 001
- Click Next, then Finish

Step 5: Add Data Source

- Click: Add Data Source, you will see several options. Choose Disk because the USB is a physical storage device. Click Next.



Step 6: Select USB Drive: Select your USB

Step 7: Configure Ingest Modules

Autopsy asks which modules to run.

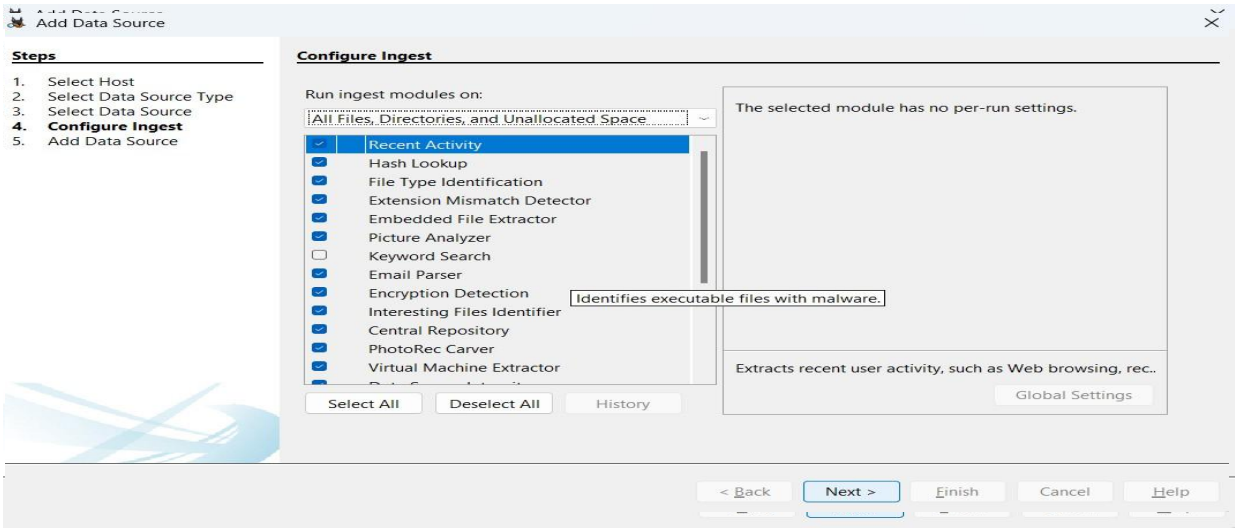
Enable:

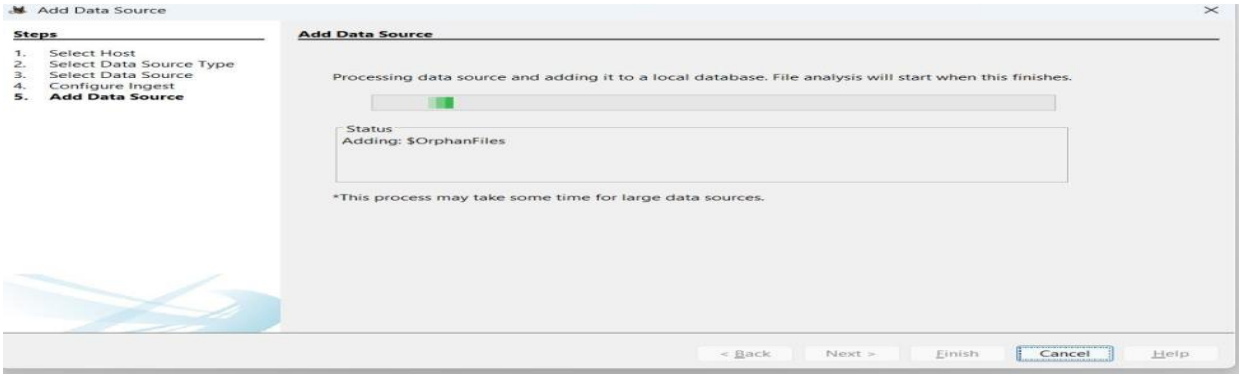
- Recent Activity
- File Type Identification
- Keyword Search (optional)
- Hash Lookup (optional)
- EXIF Parser
- Embedded File Extractor

For deleted-file recovery, File Type Identification is especially useful. Click Next.

Step 8: Start Analysis Click Finish Autopsy begins scanning the USB (This may take a few minutes)

Step 9: Wait for Processing





Step 10: Explore the Results. On the left panel, you will see sections such as:

- Data Sources
- Views
- Results
- Tags
- Reports

Expand:

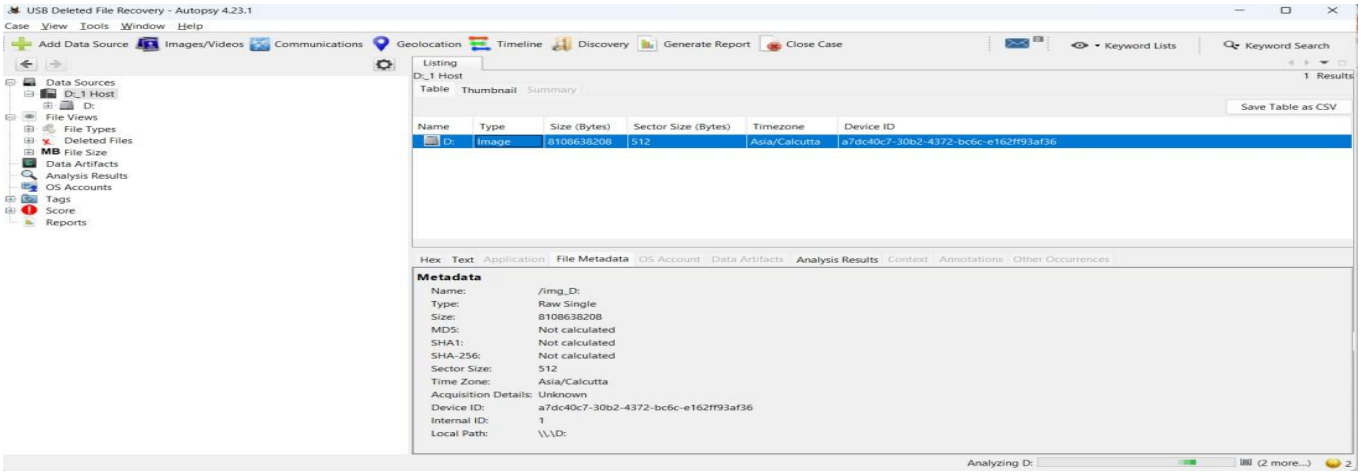
- Views → Deleted Files

This folder lists the recoverable deleted files.

Step 11: View Deleted Files Click a deleted file.

Autopsy shows:

- File name
- File size
- Created date
- Modified date
- Accessed date
- Deleted status
- Path
- Preview (for supported file types)



Step 12: Analyze Metadata Click a file and inspect:

- Creation time
- Last modified time
- Last accessed time
- Extension
- MIME type
- File size

Step 13: Recover (Extract) Deleted Files To recover a file:

- Right-click the deleted file.
- Choose Extract File(s).
- Select a destination folder on your computer, not on the USB drive.
- Click Save.

The file is now recovered.

Step 14: Click: Generate Report

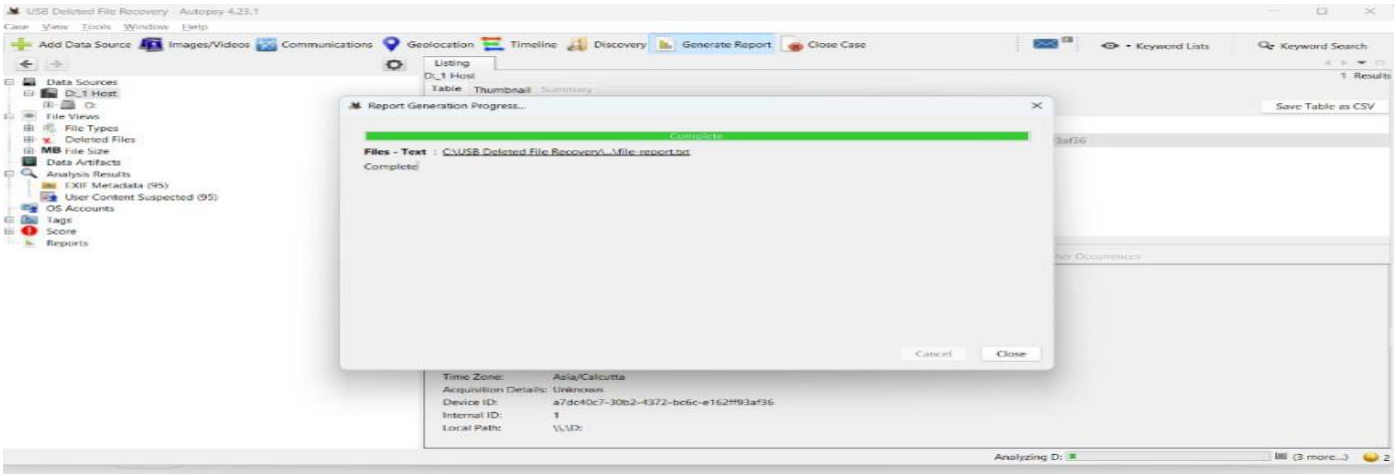
Autopsy can create a forensic report.

Choose a format such as:

- HTML
- Excel
- CSV

Include:

- Case information
- Examiner
- Recovered files
- Metadata
- Analysis summary
- Click Finish.



Result:

The deleted files from the USB drive were successfully recovered and analyzed using the Autopsy Digital Forensics Platform.

SRI ESHWAR COLLEGE OF ENGINEERING		
PARAMETERS	**MAX MARKS**	**MARKS OBTAINED**
Pre Lab	20	
Program	25	
Output	20	
Viva	10	
Total	75	
SIGNATURE OF THE FACULTY		

Ex. No: 3 Date: 13/08/2026	Implementation of Log Monitoring for Forensic Readiness

Aim:

To implement a log monitoring system that continuously monitors system/application logs, detects suspicious events, and preserves relevant log information to support digital forensic investigation and forensic readiness.

Requirements:

- Python 3.x
- VS Code / Python IDLE
- Windows or Linux operating system
- Windows Event Viewer
- Sample log file
- Python libraries:
  - time
  - re
  - datetime

Theory:

Log monitoring is an important part of digital forensic readiness because system Log monitoring is an important component of digital forensic readiness because system logs contain records of activities performed on a computer. These records may include login attempts, authentication failures, system activities, timestamps, and other security-related events.

Windows maintains security-related events in the Windows Security Event Log. By continuously monitoring these events, suspicious activities can be identified and recorded for further investigation.

In this experiment, PowerShell is used to access and monitor the Windows Security log. Event ID 4625 is used to identify failed login attempts. Multiple failed login attempts are treated as suspicious activity for the purpose of this experiment. The identified events are then exported to a CSV file so that they can be preserved and analyzed as potential forensic evidence.

Procedure:

Step 1: Open PowerShell as Administrator

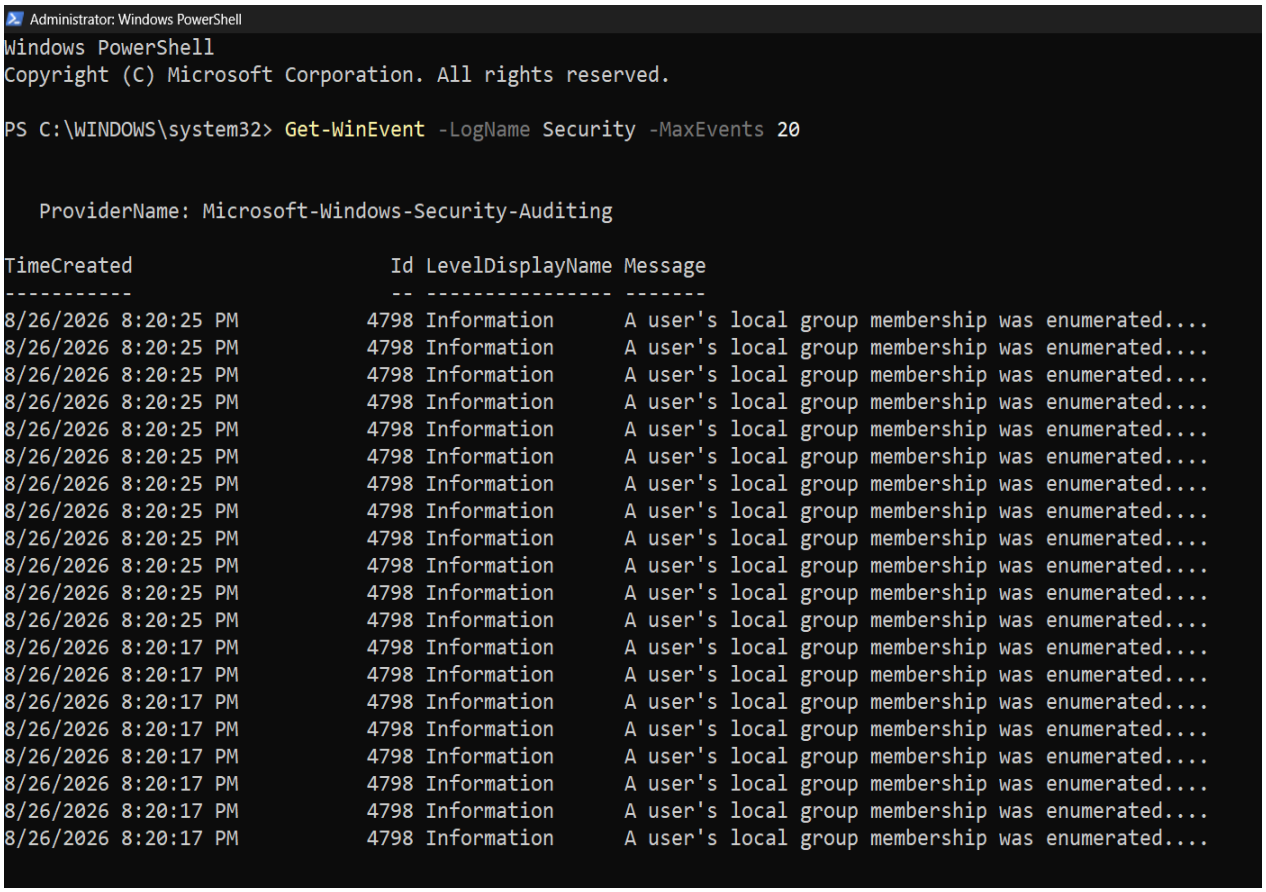
Step 2: Check the Windows Security Log Run:

- Get-WinEvent -LogName Security -MaxEvents 20

Step 3: Generate Failed Login Events

To create events for the experiment:

- Press **Windows + L** to lock your computer.
- Enter an incorrect password several times.
- For example, make **3 incorrect login attempts**.
- Enter the correct password and log in.



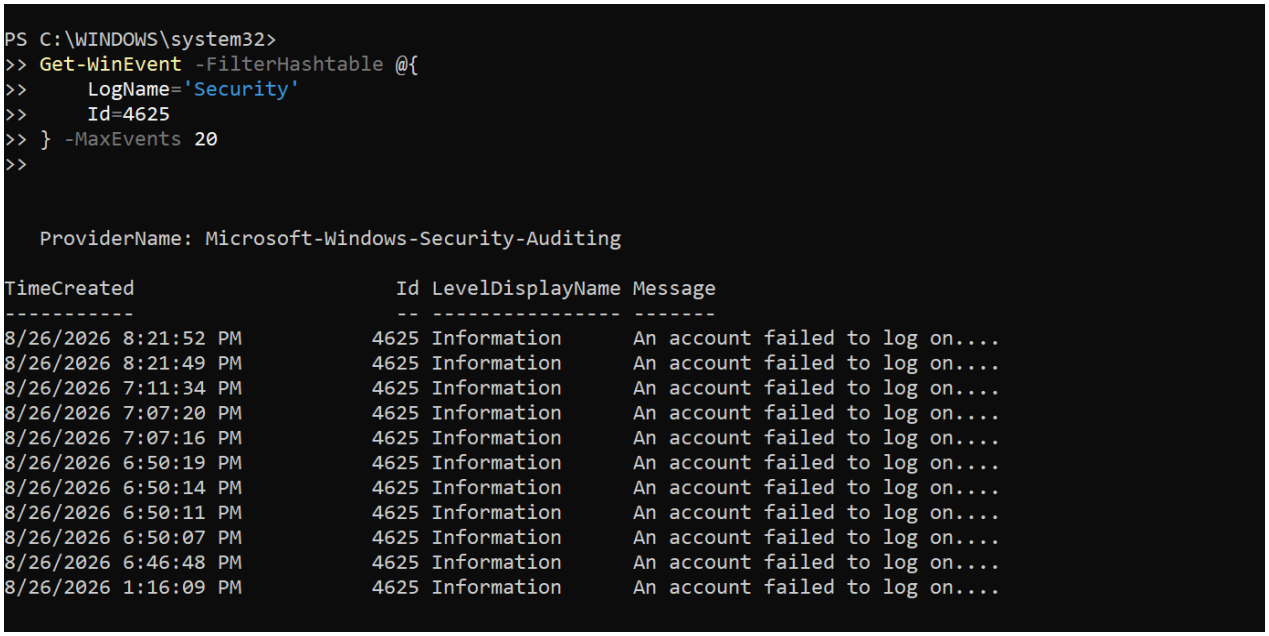
This displays the latest 20 events recorded in the Windows Security log.

Step 4: Find Failed Login Events

In PowerShell, run:

Get-WinEvent -FilterHashtable @{
LogName='Security'
Id=4625
} -MaxEvents 20

Event ID **4625** represents a failed logon event.



Step 5: Count the Failed Login Events
In PowerShell, Run:
$events = Get-WinEvent -FilterHashtable @{
LogName='Security'
Id=4625
} -MaxEvents 20
$count = $events.Count
Write-Host "Failed login events detected: $count"
if ($count -ge 3) {
Write-Host "ALERT: Multiple failed login attempts detected!"
}

```
PS C:\WINDOWS\system32>
>> $events = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     Id=4625
>> } -MaxEvents 20
>>
>> $count = $events.Count
>>
>> Write-Host "Failed login events detected: $count"
>>
>> if ($count -ge 3) {
>>     Write-Host "ALERT: Multiple failed login attempts detected!"
>> }
>>
Failed login events detected: 11
ALERT: Multiple failed login attempts detected!
```

Step 6: Monitor Continuously
In PowerShell, Run:
while ($true) {
$events = Get-WinEvent -FilterHashtable @{
LogName='Security'
Id=4625
} -MaxEvents 5 Clear-Host
Write-Host "===== FORENSIC LOG MONITOR ====="
Write-Host "Monitoring Windows Security Log" Write-Host ""
$events | Select-Object TimeCreated, Id, Message | Format-Table -Wrap
Start-Sleep -Seconds 5
}

```
Administrator: Windows PowerShell
===== FORENSIC LOG MONITOR =====
Monitoring Windows Security Log


TimeCreated            Id Message
-----------            -- -------
8/26/2026 8:21:52 PM 4625 An account failed to log on.

                         Subject:
                               Security ID:            S-1-5-18
                               Account Name:           BIBEK-DAHAL$
                               Account Domain:         WORKGROUP
                               Logon ID:               0x3E7

                         Logon Type:                   2

                         Account For Which Logon Failed:
                               Security ID:            S-1-0-0
                               Account Name:           -
                               Account Domain:         -

                         Failure Information:
                               Failure Reason:         An Error occured during Logon.
                               Status:                 0xC000006D
                               Sub Status:             0xC0000380

                         Process Information:
                               Caller Process ID:      0xb1c
                               Caller Process Name:    C:\Windows\System32\svchost.exe

                         Network Information:
                               Workstation Name:       -
                               Source Network Address: 127.0.0.1
                               Source Port:            0

                         Detailed Authentication Information:
                               Logon Process:          User32
                               Authentication Package: Negotiate
                               Transited Services:     -
                               Package Name (NTLM only):       -
                               Key Length:             0
```

Step 7: Create a Folder for Forensic Evidence

Step 8: Export the Security Events

Run:

Get-WinEvent -FilterHashtable @{ LogName='Security'
Id=4625
} -MaxEvents 20 |
Export-Csv "D:\5th Sem\Forensics_Experiment3\forensic_events.csv" – NoTypeInformation

The detected events are now exported to:

- D:\5th Sem\Forensics_Experiment3\forensic_events.csv

```
PS C:\WINDOWS\system32>
>> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     Id=4625
>> } -MaxEvents 20 |
>> Export-Csv "D:\5th Sem\Forensics_Experiment3\forensic_events_1.csv" -NoTypeInformation
>>
```

Step 9: Verify the Evidence File
Run:

- Get-Item "D:\5th Sem\Forensics_Experiment3\forensic_events.csv"

The command displays the file information.

```
PS C:\WINDOWS\system32> Get-Item "D:\5th Sem\Forensics_Experiment3\forensic_events_1.csv"


    Directory: D:\5th Sem\Forensics_Experiment3


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         8/26/2026   8:24 PM          26675 forensic_events_1.csv
```

Step 9: Verify the Evidence File
Run:

- *Get-Item "D:\5th Sem\Forensics_Experiment3\forensic_events.csv"*

The command displays the file information.

```
PS C:\WINDOWS\system32> Get-Item "D:\5th Sem\Forensics_Experiment3\forensic_events_1.csv"


    Directory: D:\5th Sem\Forensics_Experiment3

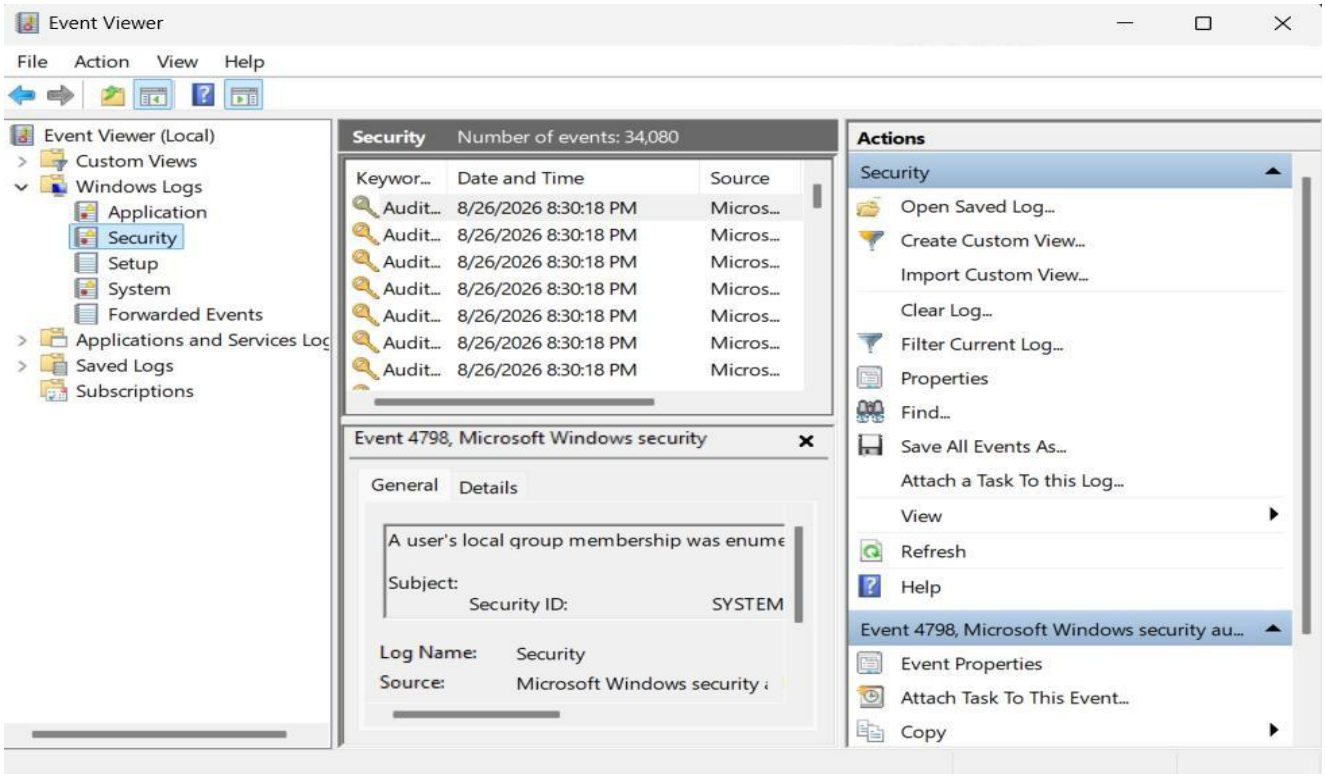

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         8/26/2026   8:24 PM          26675 forensic_events_1.csv
```

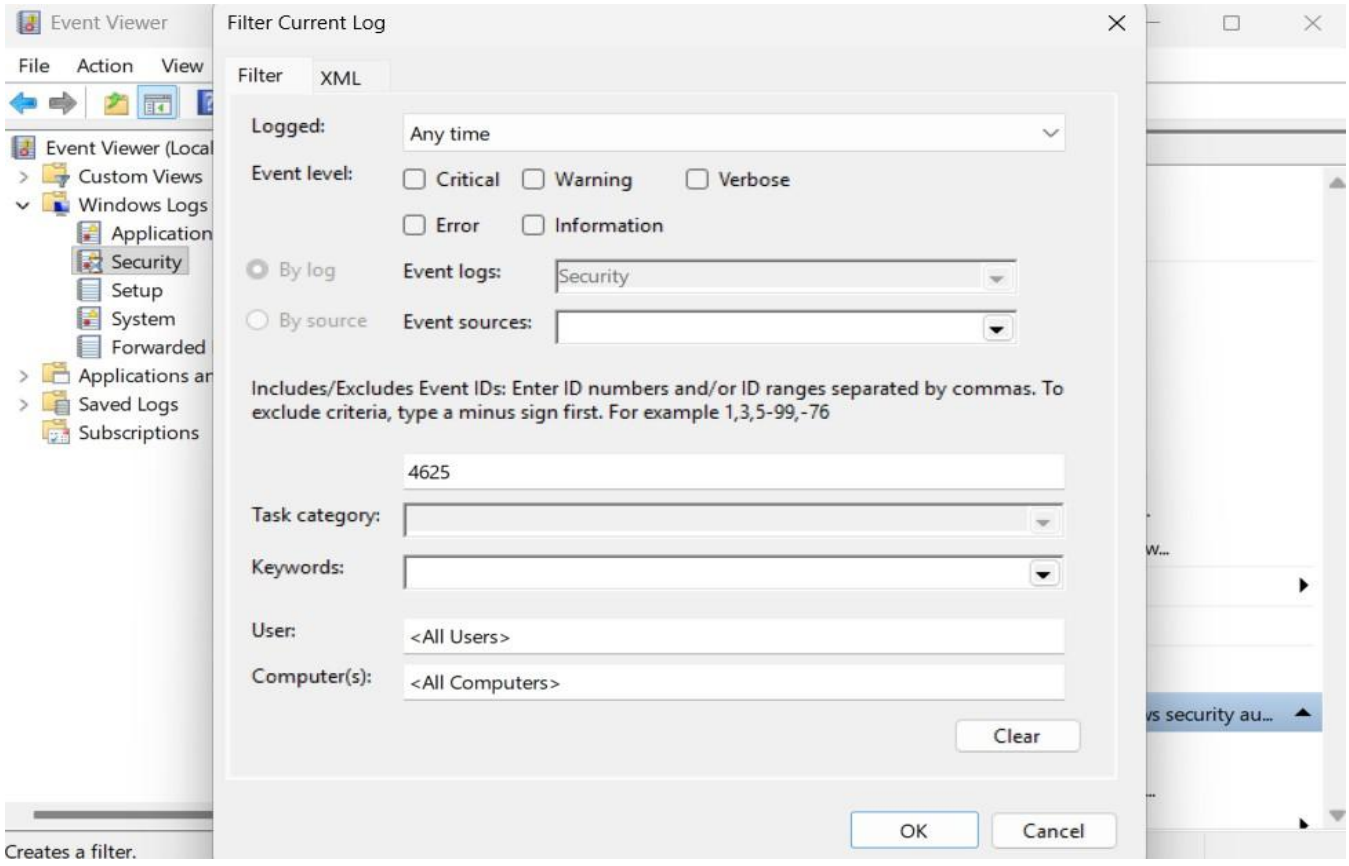
Step 10: Examine the Evidence
Open `forensic_events.csv` using Microsoft Excel. Use the information to determine:

- When the failed login occurred.
- How many failed attempts were recorded.
- Which computer generated the event.
- What security event was detected.

Step 11: Open Event Viewer for Verification
Press: Windows + R
Type: eventvwr.msc
*Press **Enter**.*
Navigate to: Event Viewer
→ Windows Logs
→ Security

Click **Filter Current Log...**
Enter: 4625





Click **OK**.
The failed login events should appear.
Double-click an event to inspect its complete details.



Output

Security Log

Time Created	ID/ Name	Message
8/27/2026 9:25:23 AM	4672 Information	An account was successfully logged on
8/27/2026 9:25:23 AM	4672 Information	An account was successfully logged on
8/27/2026 9:21:23 AM	4672 Information	An account was successfully logged on
8/27/2026 9:21:23 AM	4672 Information	An account was successfully logged on
8/27/2026 9:18:29 AM	5379 Information	Credential Manager credentials were read

Failed Login Detection

Time Created	ID/ Name	Message
8/26/2026 8:21:52 PM	4625 Information	An account failed to log on
8/26/2026 8:21:49 PM	4625 Information	An account failed to log on
8/26/2026 7:11:34 PM	4625 Information	An account failed to log on
8/26/2026 7:07:20 PM	4625 Information	An account failed to log on
8/26/2026 7:07:16 PM	4625 Information	An account failed to log on

Alert Output

Failed login events detected: 5
ALERT: Multiple failed login attempts detected!

Result

The implementation of log monitoring for forensic readiness was successfully performed using Windows PowerShell. Failed login activities were detected from the Windows Security log, suspicious activity was identified based on repeated failures, and the relevant events were exported and preserved for forensic analysis.

SRI ESHWAR COLLEGE OF ENGINEERING		
PARAMETERS	**MAX MARKS**	**MARKS OBTAINED**
Pre Lab	20	
Program	25	
Output	20	
Viva	10	
Total	75	
SIGNATURE OF THE FACULTY		

Ex. No: 4 Date: 03/09/2026	Analysis of Data Extraction from an iOS Backup

Aim:

To analyze an iOS device backup and extract forensic information such as contacts, messages, call history, application data, and metadata without requiring a physical iPhone.

Requirements:

Hardware

- Computer/Laptop
- Digital image file (JPG/JPEG captured using a smartphone or digital camera)

Software

- Windows or Linux Operating System
- iLEAPP (iOS Logs, Events, And Plists Parser) for extracting and analyzing iOS backup artifacts
- Python 3 to run iLEAPP.
- Command Prompt or PowerShell.
- A sample iOS backup or authorized forensic dataset.

Theory:

An iOS backup contains data from an iPhone or iPad, including application data, messages, contacts, call history, settings, and other device artifacts, depending on the backup type and configuration. In digital forensics, analyzing an iOS backup helps investigators recover and interpret relevant digital evidence. iOS backups may contain databases (SQLite), property list files (PLIST), application files, and media such as photos and videos. Tools such as iLEAPP (iOS Logs, Events, And Plists Parser) help extract and analyze supported artifacts from iOS backups and filesystem extractions.

In digital forensics, image metadata helps an investigator to:

- Recover digital artifacts: Examine available messages, contacts, call records, browsing activity, and application data.
- Establish a timeline: Analyze timestamps and records to understand the sequence of events.
- Identify application activity: Examine supported application databases, logs, and configuration files.
- Support investigations: Correlate multiple artifacts to understand relevant user or device activity.

To preserve evidence integrity, investigators should retain the original iOS backup, analyze a verified copy, record and verify its SHA-256 hash, and document all acquisition and analysis procedures

Algorithm:

1. Download and install ExifTool.
2. Start the Android emulator using Android Studio.
3. Verify the ADB installation and emulator connection.
4. Retrieve the Android device information using ADB commands.
5. Create a sample text file on the computer.
6. Transfer the sample file to the Android emulator using the `adb push` command.
7. Verify the presence of the file on the emulator.
8. Extract the file from the emulator to the computer using the `adb pull` command.
9. Calculate the SHA-256 hashes of the original and extracted files.
10. Compare both hash values to verify file integrity.
11. End

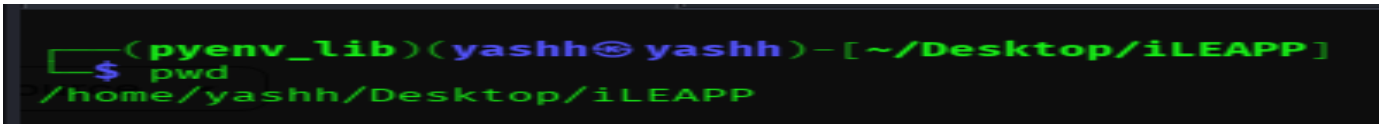
Procedure:

Step 1: Install iLEAPP

- Download iLEAPP from the official GitHub repository.
- Install Python 3 and the required dependencies.
- Extract the iLEAPP ZIP file into a working folder, for example C:\iOSForensics\iLEAPP.

Verify the installation by opening Command Prompt in that folder and running:

- cd iLEAPP



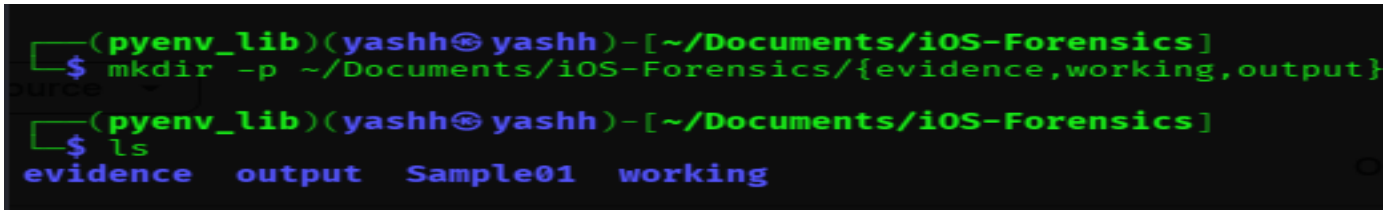
Prepare the iOS Backup

- Obtain an authorized iOS backup or public forensic sample dataset.
- Copy the backup into C:\iOSForensics\evidence without modifying the original.

Create a separate working copy for analysis.

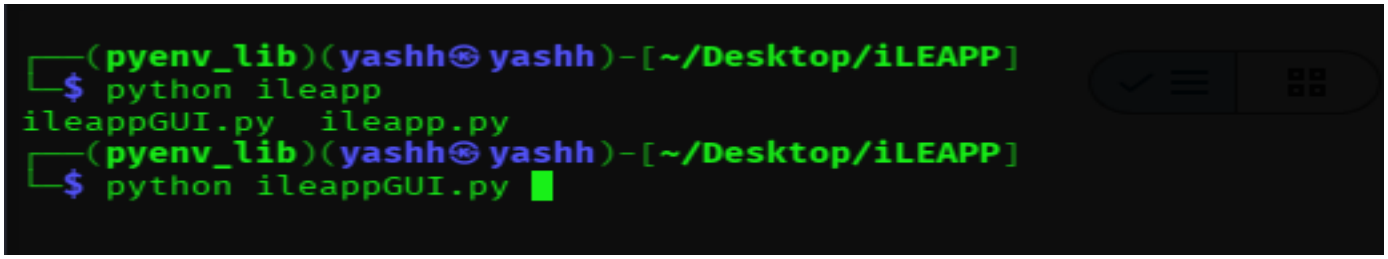
Step 2: Create the Evidence and Output Folders

Create separate folders to preserve the original dataset, analyze a working copy, and store the results.



Step 2: Launch the iLEAPP Application

- Open the terminal and navigate to the iLEAPP directory.
- Activate the Python virtual environment if required.
- Run the following commands to launch the application:



- The iLEAPP graphical user interface will open. Click the input selection button and select the downloaded iOS forensic image or supported backup.
- Select the appropriate input type and specify the output folder as ~/Documents/iOS-Forensics/output. Click the option to start processing and wait for the analysis to finish.
- Once processing is complete, open the generated HTML report from the output folder and examine the extracted forensic artifacts.

Step 5: Analyze the Extracted Data

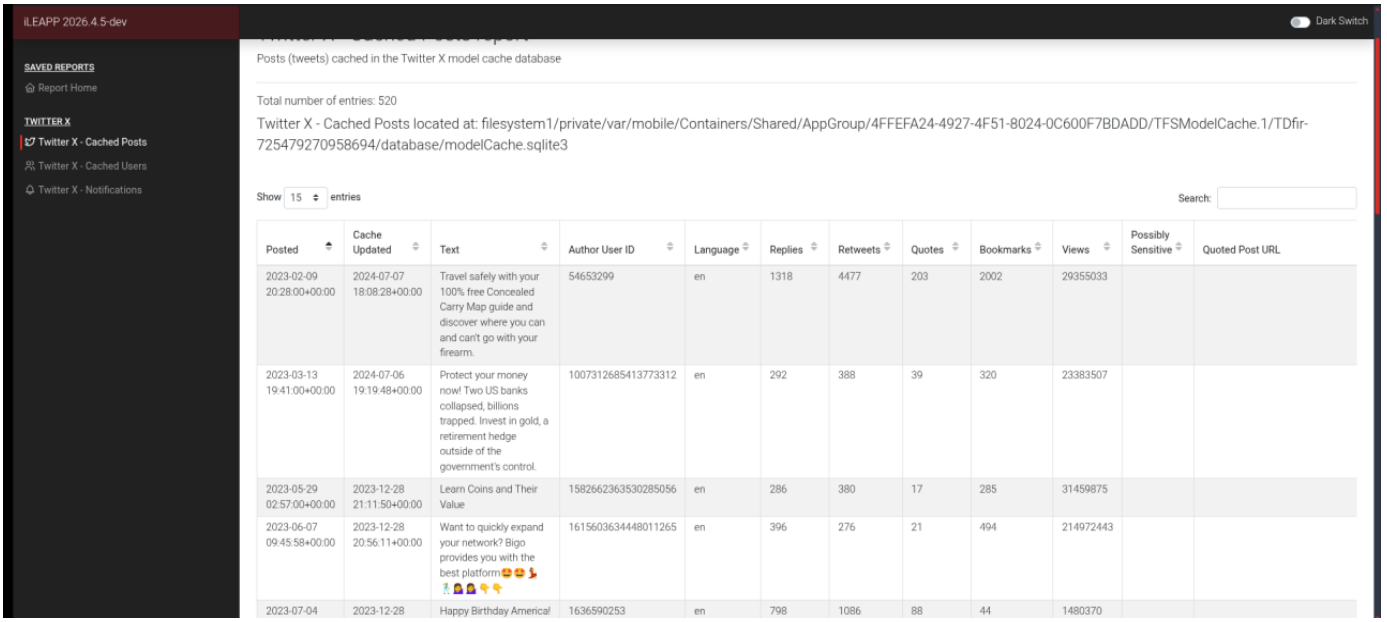
- After processing is complete, open the output folder selected in the iLEAPP graphical user interface.
- Open the generated HTML report in a web browser.
- Examine the extracted forensic artifacts, including messages, contacts, application data, logs, and timestamps, where available.

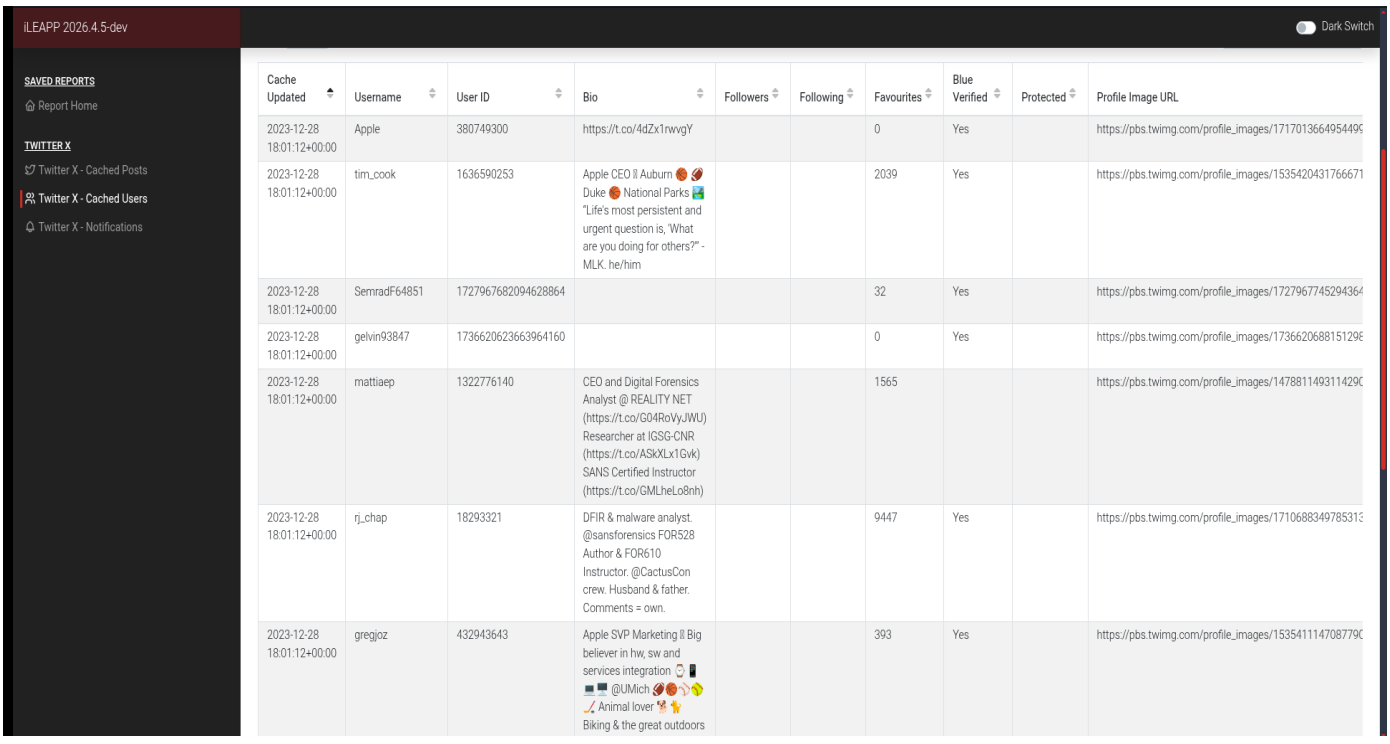
- Review the relevant records and identify information useful for the investigation.
- Record the findings, including artifact details and timestamps, in the forensic report.

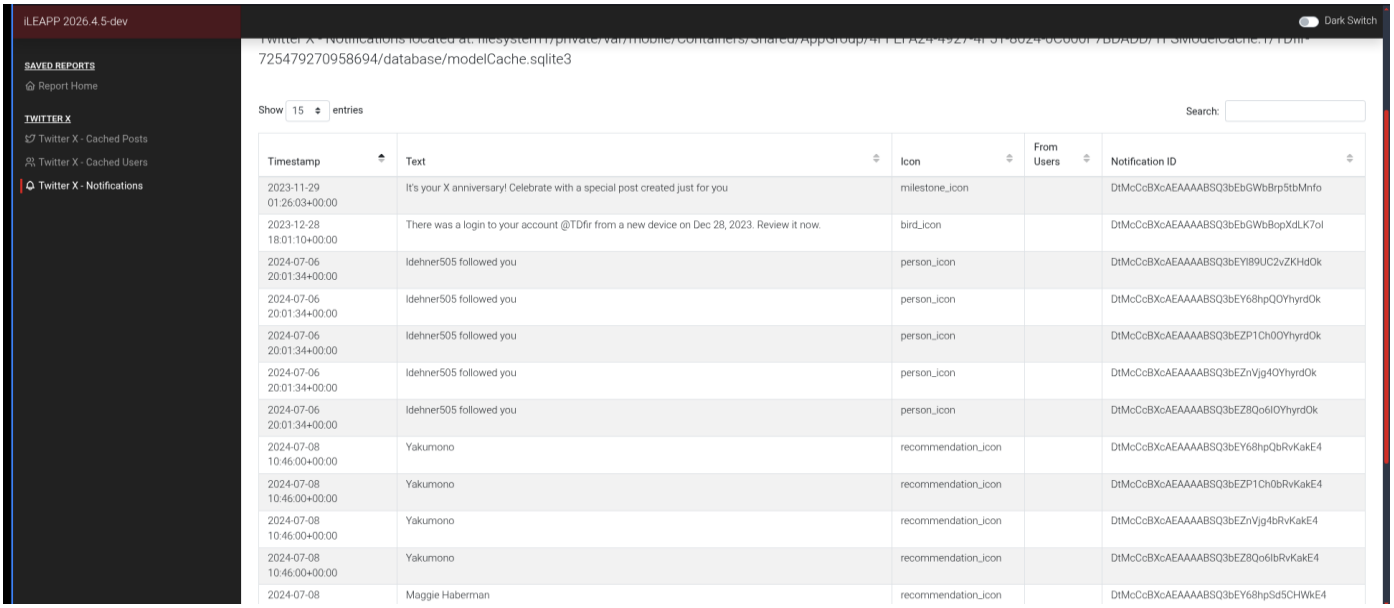


Step 6: Verify Evidence Integrity and View the Report

After analysis, calculate the hash again.







Result:

The iOS device backup was successfully analyzed, and forensic information such as contacts, messages, call history, and application data was extracted. The extracted data was examined to identify relevant digital evidence without accessing the physical iPhone.

SRI ESHWAR COLLEGE OF ENGINEERING		
PARAMETERS	**MAX MARKS**	**MARKS OBTAINED**
Pre Lab	20	
Program	25	
Output	20	
Viva	10	
Total	75	
SIGNATURE OF THE FACULTY		

Ex. No: 5 Date: 01/10/2026	**Implementation of Data Extraction from an Android Device using ADB**

Aim:

To extract a file from an Android emulator using Android Debug Bridge (ADB) and verify the integrity of the extracted file using the SHA-256 hashing algorithm.

Requirements:

Software

- Windows 10/11 or Linux (Ubuntu/Kali)
- Android Studio software.
- Android Debug Bridge (ADB) platform tools.

Theory:

Android Debug Bridge (ADB) is a command-line tool that enables communication between a computer and an Android device or emulator. It consists of three components: the ADB client, ADB server, and ADB daemon (adbd). In digital forensics, ADB is used to retrieve device information, execute permitted shell commands, and transfer files for examination. The adb devices command identifies connected devices, adb push transfers files from the computer to the Android device, and adb pull extracts files from the device to the computer. In this experiment, an Android emulator is used to demonstrate file transfer and extraction. The SHA-256 cryptographic hash algorithm is then applied to the original and extracted files to compare their hash values and verify file integrity. Matching hash values indicate that the file contents are identical with respect to the SHA-256 comparison. Proper documentation, authorization, and preservation of original evidence are essential for maintaining forensic reliability.

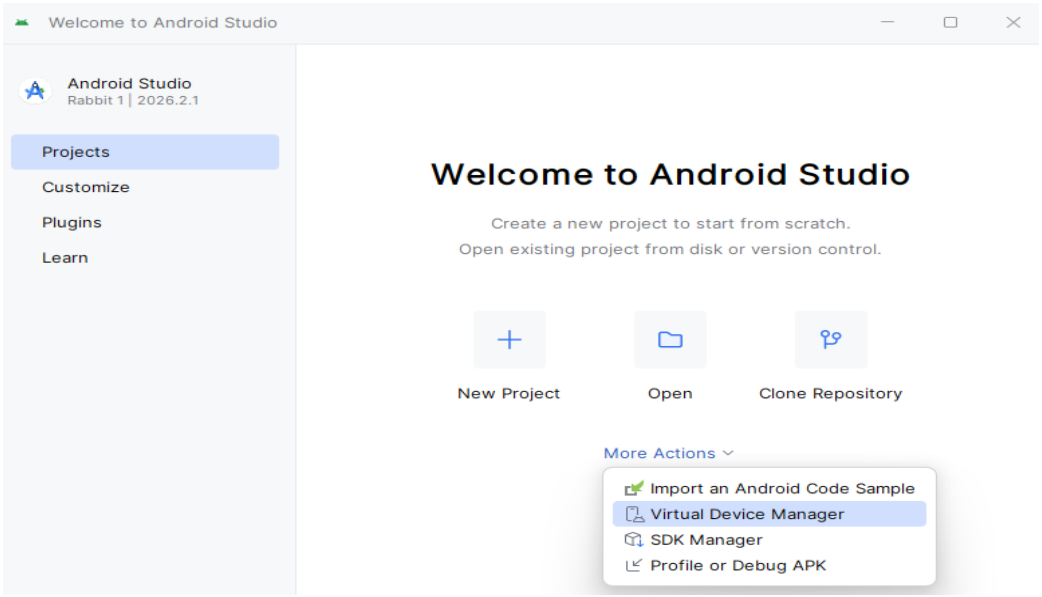
Algorithm:

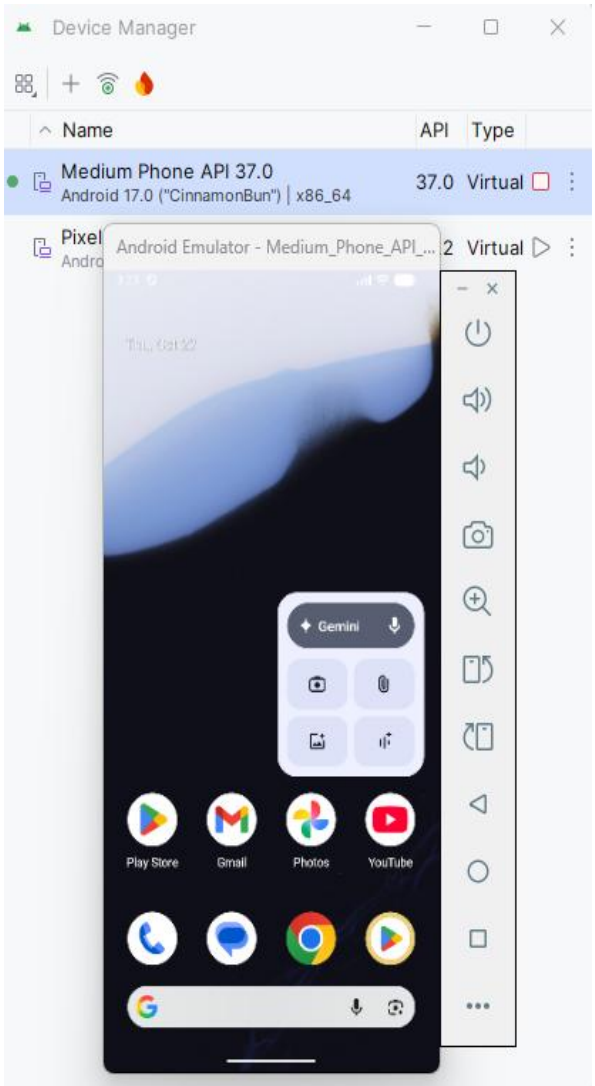
1. Download and install ExifTool.
2. Start the Android emulator using Android Studio.
3. Verify the ADB installation and emulator connection.
4. Retrieve the Android device information using ADB commands.
5. Create a sample text file on the computer.
6. Transfer the sample file to the Android emulator using the `adb push` command.
7. Verify the presence of the file on the emulator.
8. Extract the file from the emulator to the computer using the `adb pull` command.
9. Calculate the SHA-256 hashes of the original and extracted files.
10. Compare both hash values to verify file integrity.
11. End

Procedure:

Step 1: Start the Android Emulator

- Open Android Studio.
- Navigate to Tools → Device Manager.
- Create a virtual Android device if one is not available.
- Start the emulator.


Welcome to Android Studio
Android Studio
Rabbit 1 | 2026.2.1
Projects
Customize
Plugins
Learn
Welcome to Android Studio
Create a new project to start from scratch.
Open existing project from disk or version control.
New Project
Open
Clone Repository
More Actions
Import an Android Code Sample
Virtual Device Manager
SDK Manager
Profile or Debug APK


Device Manager
Name
API
Type
Medium Phone API 37.0
Android 17.0 ("CinnamonBun") | x86_64
37.0 Virtual
Pixel
Virtual
Android Emulator - Medium_Phone_API_...
Gemini
Play Store
Gmail
Photos
YouTube

Step 2: Verify ADB Installation

Open Command Prompt and execute:

- adb version

This command displays the installed ADB version.

Change directory if needed: cd C:\Users\dhamu\AppData\Local\Android\Sdk\platform-tools

Step 3: Check Device Connection

Execute: adb devices

Verify that the emulator appears in the device list with the status device.

Step 4: Collect Device Information

Execute the following commands:

- adb shell getprop ro.build.version.release
- adb shell getprop ro.product.model
- adb shell getprop ro.product.manufacturer

These commands retrieve the Android version, device model, and manufacturer.

```
Microsoft Windows [Version 10.0.26200.9711]
(c) Microsoft Corporation. All rights reserved.

C:\Users\dhamu>cd C:\Users\dhamu\AppData\Local\Android\Sdk\platform-tools

C:\Users\dhamu\AppData\Local\Android\Sdk\platform-tools>adb version
Android Debug Bridge version 1.0.41
Version 37.0.1-15733141
Installed as C:\Users\dhamu\AppData\Local\Android\Sdk\platform-tools\adb.exe
Running on Windows 10.0.26200

C:\Users\dhamu\AppData\Local\Android\Sdk\platform-tools>adb devices
List of devices attached
emulator-5554   device


C:\Users\dhamu\AppData\Local\Android\Sdk\platform-tools>adb shell getprop ro.build.version.release
17

C:\Users\dhamu\AppData\Local\Android\Sdk\platform-tools>adb shell getprop ro.product.model
sdk_gphone64_x86_64

C:\Users\dhamu\AppData\Local\Android\Sdk\platform-tools>adb shell getprop ro.product.manufacturer
Google

C:\Users\dhamu\AppData\Local\Android\Sdk\platform-tools>
```

Step 5: Prepare the USB Drive

Create a Sample File

Execute:

mkdir "%USERPROFILE%\Documents\MobileForensicsLab"

echo Digital Forensics Lab Sample > "%USERPROFILE%\Documents\MobileForensicsLab\sample.txt"

This creates a sample text file on the computer.

Step 6: Transfer the File to the Emulator

Execute:

adb push "%USERPROFILE%\Documents\MobileForensicsLab\sample.txt" /sdcard/Download/sample.txt

The file is copied from the computer to the emulator.

Step 7: Verify the File on the Emulator

Execute:

adb shell ls -l /sdcard/Download/sample.txt

adb shell cat /sdcard/Download/sample.txt

These commands verify the file's presence and display its contents.

Step 8: Extract the File from the Emulator

Execute:

adb pull /sdcard/Download/sample.txt
"%USERPROFILE%\Documents\MobileForensicsLab\extracted_sample.txt"

The file is copied back from the emulator to the computer.

Step 9: Verify File Integrity

Calculate the SHA-256 hash of the original file:

certutil -hashfile "%USERPROFILE%\Documents\MobileForensicsLab\sample.txt" SHA256

Calculate the SHA-256 hash of the extracted file:

certutil -hashfile "%USERPROFILE%\Documents\MobileForensicsLab\extracted_sample.txt" SHA256

Compare the two hash values. If both values match, the original and extracted files have identical contents with respect to the SHA-256 comparison.

Result:

The file was successfully extracted from the Android emulator using ADB. Its integrity was verified using the SHA-256 hash, confirming that the extracted file remained unchanged.

SRI ESHWAR COLLEGE OF ENGINEERING		
PARAMETERS	**MAX MARKS**	**MARKS OBTAINED**
Pre Lab	20	
Program	25	
Output	20	
Viva	10	
Total	75	
SIGNATURE OF THE FACULTY		